

SimuFragDB — Simulating a Distributed RDBMS

Repo Link: github.com/Rohitangshu2026/NoSQL

Introduction

SimuFragDB is a simulated distributed relational database system built on PostgreSQL to emulate a multi-node database architecture on a single machine. Each database instance represents an independent fragment that stores a horizontal partition of the overall dataset.

The system demonstrates horizontal fragmentation, deterministic query routing, fragment-local execution, and application-level aggregation across fragments. The primary objective of this assignment is to explore distributed database concepts while focusing on architectural clarity and correct distributed execution rather than strict consistency or fault tolerance.

System Architecture

The system consists of one baseline database for correctness verification and multiple fragment databases (fragment0, fragment1, fragment2). Each fragment maintains the same schema but stores a subset of rows based on horizontal partitioning. The baseline database is used to generate expected_output.txt for validation.

A deterministic routing function assigns data to fragments:

$$fragmentId = |student_id.hashCode()| \bmod numFragments$$

This ensures that each student record resides in exactly one fragment, related grade records are routed consistently, and cross-fragment joins are avoided. All queries are routed to fragments through a routing component.

Database Schema

Every fragment contains an identical schema consisting of:

- **Student** — primary entity storing student details
- **Grade** — mapping between students and courses with scores
- **Course** — replicated lookup table

The Course table is replicated across all fragments to support local aggregation without requiring distributed joins.

Query Execution Strategy

All modification operations are fragment-aware. Inserts, updates, and deletions are routed using student_id to ensure fragment-local execution.

Supported write operations include:

INSERT_STUDENT, INSERT_GRADE, UPDATE_GRADE, DELETE_STUDENT_COURSE.

Read operations include:

READ_PROFILE, READ_SCORE, READ_ALL.

Global analytical queries use a two-phase aggregation strategy.

Average Score by Department

Each fragment computes local aggregates SUM(score) and COUNT(*) grouped by department. The application then merges these results to compute global averages using:

$$Average = \frac{Total\ Score}{Total\ Count}$$

No cross-fragment joins are used.

Students with Maximum Courses

Each fragment identifies the local maximum number of course enrollments. The coordinator then determines the global maximum and returns students matching this value. Results are sorted by student_id to ensure deterministic output.

Workload Execution

Commands are read from workload.txt and executed sequentially by the driver component. Two runs are performed:

- Baseline run → expected_output.txt
- Distributed run → output.txt

Both outputs are formatted identically to allow direct comparison.

Accuracy Evaluation

Correctness is validated by comparing the distributed output with the baseline output line by line.

Result:

$$Accuracy = 100\%$$

Total number of query output lines: 2462

The distributed implementation perfectly matches the baseline execution.

Performance Measurement

Measured workload execution time:

$$709\ ms$$

This demonstrates efficient routing and aggregation across fragments.

Implementation Challenges

- Designing aggregation without cross-fragment joins
- Maintaining deterministic routing
- Matching floating-point precision with baseline output

Conclusion

SimuFragDB successfully simulates distributed database behavior using horizontal fragmentation and deterministic routing. The system achieves correct execution, efficient aggregation, and perfect output matching while remaining architecturally simple.